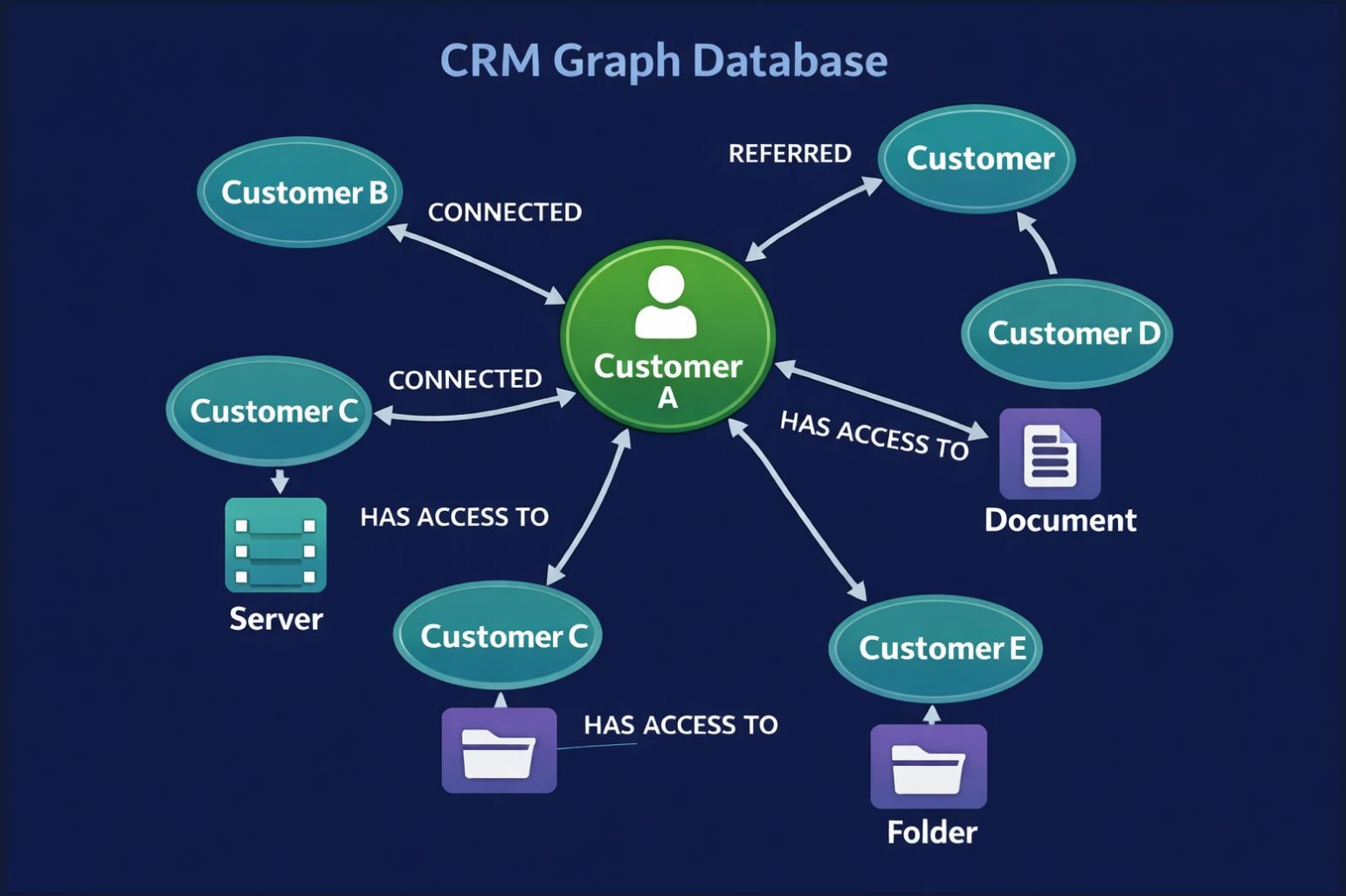
AIRFLOW SUMMIT

Orchestrating Graph Database Workloads in Apache Airflow with Apache TinkerPop

Ahmad Farhan

Why graph databases?

- Relationships
- Vertices (or nodes) & Edges
- Queries are traversal-based, not join
- Fit for:
  - CRM
  - Medical diagnosis
  - Supply-chain
- Knowledge graphs within AI era


CRM Graph Database
Customer B
CONNECTED
REFERRED
Customer
Customer A
Customer D
Customer C
CONNECTED
HAS ACCESS TO
Document
Server
HAS ACCESS TO
Customer C
Customer E
HAS ACCESS TO
Folder

What is Apache TinkerPop™?

- Vendor neutral graph framework
- Managed Services
  - AWS Neptune
  - Azure Cosmos DB
- Gremlin traversal language as a query language
- Works with any gremlin server

The Problem in Practice

- CustomHook scripts
- Python Packages dependencies
  - Use @task.virtualenv decorator
- SQL API exist but not fit for purpose

```python
class GremlinConnectionHook(BaseHook):
    conn_type = 'gremlin'
    default_port = 443

    def __init__(self, gremlin_conn_id='gremlin_default'):
        self.gremlin_conn_id = gremlin_conn_id

    def _get_conn_id(self):
        conn = BaseHook.get_connection(self.gremlin_conn_id)
        if conn is None:
            raise Exception(f"Connection {self.gremlin_conn_id} not found.")
        return conn
```

```python
@task.virtualenv(
        task_id="ingest_gremlin_data",
        requirements=["gremlinpython==3.7.0"],
        system_site_packages=True)
def ingest_gremlin_data(business_date: str):
    try:
        from gremlin_helper import GremlinConnectionHook

        hook = GremlinConnectionHook('GREMLIN')
        query="g.V()"
        result = hook.get_results_df(gremlin_query=query)
```

Why I built this provider?

- Create **out of the box** connector
- Have a production ready hook
- Goal: Make gremlin supported graph databases first-class citizens in Airflow
  - Graph loads & updates
  - Graph analytics
  - Feature generation in AI

Core Components

- Provider building blocks
  - GremlinHook
  - GremlinOperator
  - Airflow Connection Type
- Fully tested – unit & integration testing

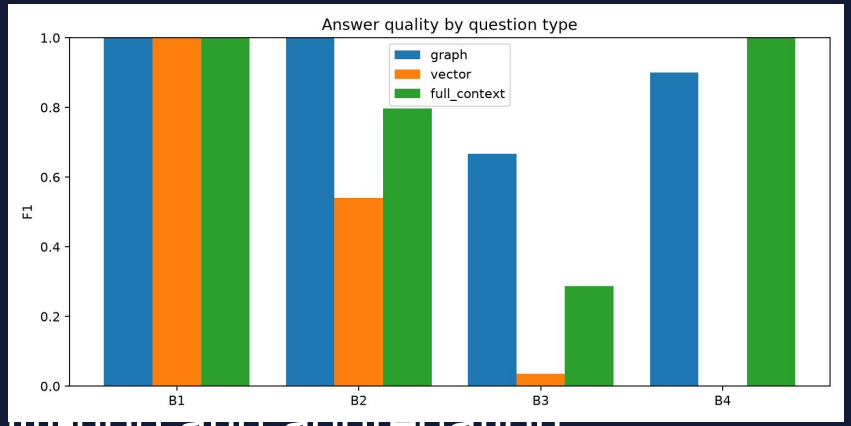
```python
@task
def customers_by_tier() -> dict[str, int]:
    """The hook, for when you want to do something with the result.

    ``GremlinHook.run()`` opens the connection, submits, and closes it
    again -- so this is the whole round trip in one line.
    """
    hook = GremlinHook(conn_id=GREMLIN_CONN)
    rows = hook.run("g.V().hasLabel('Customer').groupCount().by('tier')")
```

Demo Overview

1. Act 1 — the provider
   a. Gremlin_hello
   b. DAG → hook → query → results in Airflow
2. Act 2 — what that unlocks
   a. 215 support tickets, no schema, no graph
   b. kg_build @task.llm reads them → 65 relationships → Gremlin
   c. vector_build same tickets → embeddings → pgvector
   d. rag_showdown_live one question, both indexes, scored

   One corpus. Two indexes. One orchestrator.

Lessons Learned



- Graph wins multi-hop and aggregation.
  - Vectors win lookup and prose — outright.
  - Hybrid, not either.
- Observability is the biggest win.

What's Next?

- Contributors are welcomed for the following:
  - GremlinOperator has no bindings' parameter at all.
  - GremlinHook.run() closes the client on EVERY call.
  - Driver implementation

```python
from gremlin_python.driver.driver_remote_connection import DriverRemoteConnection
from gremlin_python.process.anonymous_traversal import traversal

remote_conn = DriverRemoteConnection('ws://localhost:8182/gremlin', 'g')

g = traversal().withRemote(remote_conn)

gr = g.V().hasLabel('Customer')
print(gr.toList())

remote_conn.close()
```

Questions?

/atfarhan

Docs

/ahmadtfarhan